

**LAPORAN PROJEK AKHIR MATA KULIAH KECERDASAN BUATAN
UNIB MAPS – IMPLEMENTASI ALGORITMA DIJKSTRA PADA SISTEM
NAVIGASI KAMPUS UNIVERSITAS BENGKULU**



Disusun Oleh :
Ariel Tiansyahrullah

Kelas :
A

Dosen Pengampu :
Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas segala rahmat dan karunia-Nya sehingga saya dapat menyelesaikan laporan proyek yang berjudul **“UNIB Maps – Implementasi Algoritma Dijkstra pada Sistem Navigasi Kampus Universitas Bengkulu”** dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban terhadap pengembangan aplikasi navigasi kampus berbasis web yang memanfaatkan algoritma shortest path untuk membantu pengguna menemukan rute terpendek menuju lokasi yang diinginkan di lingkungan Universitas Bengkulu.

Perkembangan teknologi informasi telah memberikan banyak kemudahan dalam berbagai aspek kehidupan, termasuk dalam bidang navigasi dan pencarian rute. Dengan memanfaatkan teknologi pemetaan digital dan algoritma pencarian jalur terpendek, pengguna dapat memperoleh informasi rute yang lebih efektif dan efisien. Oleh karena itu, pada proyek ini dikembangkan sebuah sistem navigasi kampus yang mampu menampilkan lokasi-lokasi penting di Universitas Bengkulu serta menentukan jalur terpendek berdasarkan algoritma Dijkstra.

Saya menyadari bahwa laporan ini masih memiliki berbagai kekurangan baik dari segi penyajian maupun isi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan guna penyempurnaan laporan ini di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi referensi dalam pengembangan sistem navigasi berbasis algoritma pencarian jalur terpendek.

Bengkulu, Mei 2026

Penyusun

DAFTAR ISI

COVER MAKALAH.....	i
KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	2
BAB II LANDASAN TEORI	3
2.1 Sistem Informasi Geografis (SIG)	3
2.2 Teori Graf.....	3
2.3 Algoritma Dijkstra	4
2.4 OpenStreetMap (OSM).....	4
2.5 Leaflet.js.....	5
2.6 Open Source Routing Machine (OSRM).....	5
BAB III METODE PENELITIAN.....	7
3.1 Analisis Kebutuhan Sistem	7
3.2 Pengumpulan Data	7
3.3 Perancangan Graf.....	7
3.4 Perancangan Sistem	7
3.5 Implementasi Algoritma Dijkstra	8
3.6 Pengujian Sistem.....	8
BAB IV HASIL DAN PEMBAHASAN	9

4.1	Pembangunan Sistem Navigasi Berbasis Web	9
4.2	Representasi Lokasi Kampus dalam Bentuk Graf	10
4.3	Pembentukan Struktur Graph.....	12
4.4	Implementasi Algoritma Dijkstra	13
4.5	Visualisasi Jalur Terpendek Pada Peta Digital	15
4.6	Implementasi Mode Transportasi	17
4.7	Hasil Pengujian Sistem	18
4.8	Analisis Hasil	20
4.8.1	Analisis Kinerja Algoritma Dijkstra	20
4.8.2	Analisis Pengaruh Mode Transportasi	21
4.8.3	Analisis Visualisasi Rute	21
4.8.4	Kelebihan dan Keterbatasan Sistem.....	22
BAB V PENUTUP.....		24
5.1	Kesimpulan.....	24
5.2	Saran.....	24
DAFTAR PUSTAKA.....		26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Universitas Bengkulu merupakan salah satu perguruan tinggi yang memiliki area kampus cukup luas dengan berbagai fasilitas dan gedung yang tersebar di beberapa lokasi. Banyaknya gedung fakultas, laboratorium, perpustakaan, pusat layanan, serta fasilitas umum lainnya sering kali menyebabkan mahasiswa baru maupun pengunjung mengalami kesulitan dalam menemukan lokasi tujuan secara cepat dan efisien. Proses pencarian lokasi secara manual tidak jarang mengakibatkan pengguna menempuh jalur yang lebih jauh dibandingkan jalur yang sebenarnya dapat dilalui. Perkembangan teknologi pemetaan digital memungkinkan penyediaan informasi lokasi secara interaktif melalui aplikasi berbasis web. Dengan memanfaatkan peta digital dan sistem navigasi, pengguna dapat memperoleh informasi mengenai posisi suatu lokasi serta rute yang harus ditempuh menuju tujuan tertentu. Namun, agar sistem navigasi dapat memberikan rekomendasi rute yang optimal, diperlukan suatu algoritma yang mampu menentukan jalur terpendek dari titik asal menuju titik tujuan.

Salah satu algoritma yang banyak digunakan untuk menyelesaikan permasalahan pencarian jalur terpendek adalah algoritma Dijkstra. Algoritma ini bekerja dengan menghitung jarak minimum dari suatu titik awal ke seluruh titik lain dalam suatu graf berbobot hingga ditemukan jalur dengan total bobot terkecil menuju tujuan. Pada proyek ini, lokasi-lokasi penting di lingkungan Universitas Bengkulu direpresentasikan sebagai simpul (node), sedangkan jalur penghubung antar lokasi direpresentasikan sebagai sisi (edge) yang memiliki bobot berupa jarak tempuh. Berdasarkan permasalahan tersebut, dikembangkan sebuah aplikasi berbasis web bernama UNIB Maps – Dijkstra Navigator yang mengintegrasikan teknologi OpenStreetMap, Leaflet, dan Open Source Routing Machine (OSRM) untuk menampilkan peta kampus secara interaktif. Sistem ini memanfaatkan

algoritma Dijkstra untuk menentukan jalur terpendek antar lokasi di lingkungan Universitas Bengkulu sehingga pengguna dapat memperoleh rute yang lebih efektif sesuai dengan moda transportasi yang dipilih, yaitu berjalan kaki, sepeda motor, atau mobil.

1.2 Rumusan Masalah

- 1 Bagaimana membangun sistem navigasi kampus Universitas Bengkulu berbasis web menggunakan Leaflet dan OpenStreetMap?
- 2 Bagaimana merepresentasikan lokasi dan jalur di lingkungan Universitas Bengkulu ke dalam bentuk graf berbobot?
- 3 Bagaimana membangun struktur graph dari data node dan edge agar dapat diproses oleh algoritma Dijkstra?
- 4 Bagaimana menerapkan algoritma Dijkstra untuk menentukan jalur terpendek antara titik asal dan tujuan?
- 5 Bagaimana menampilkan hasil pencarian jalur terpendek secara interaktif pada peta digital?
- 6 Bagaimana mengimplementasikan mode transportasi yang memengaruhi pemilihan jalur pada sistem navigasi?

1.3 Tujuan

1. Merancang dan membangun aplikasi navigasi kampus berbasis web untuk Universitas Bengkulu.
2. Merepresentasikan lokasi-lokasi penting kampus ke dalam struktur graf berbobot.
3. Mengimplementasikan algoritma Dijkstra untuk menentukan jalur terpendek dari titik asal ke titik tujuan.
4. Menampilkan rute hasil perhitungan algoritma pada peta digital secara interaktif.
5. Membantu mahasiswa, dosen, dan pengunjung dalam menemukan lokasi tujuan dengan lebih cepat dan efisien.

BAB II

LANDASAN TEORI

2.1 Sistem Informasi Geografis (SIG)

Sistem Informasi Geografis (SIG) merupakan sistem berbasis komputer yang digunakan untuk mengumpulkan, menyimpan, mengelola, menganalisis, dan menampilkan data yang memiliki informasi geografis atau lokasi tertentu. SIG memungkinkan pengguna untuk melihat hubungan antara data dan posisi geografisnya sehingga dapat membantu dalam pengambilan keputusan yang berkaitan dengan suatu wilayah (Burrough & McDonnell, 1998).

Pada penelitian ini, SIG digunakan sebagai dasar dalam pengembangan sistem navigasi kampus Universitas Bengkulu. Data lokasi gedung dan fasilitas kampus ditampilkan pada peta digital sehingga pengguna dapat melihat posisi lokasi dan memperoleh informasi jalur menuju tujuan yang diinginkan.

2.2 Teori Graf

Graf merupakan struktur data yang terdiri dari simpul (*node* atau *vertex*) dan sisi (*edge*) yang menghubungkan antar simpul. Graf digunakan untuk merepresentasikan hubungan antar objek sehingga dapat digunakan dalam berbagai permasalahan, seperti jaringan komputer, transportasi, dan pencarian rute (Gross & Yellen, 2006).

Dalam sistem navigasi, setiap lokasi dapat direpresentasikan sebagai node, sedangkan jalur yang menghubungkan lokasi tersebut direpresentasikan sebagai edge. Setiap edge dapat memiliki bobot (*weight*) yang menunjukkan nilai tertentu, seperti jarak, waktu tempuh, atau biaya perjalanan. Graf yang memiliki bobot pada setiap edge disebut sebagai graf berbobot (*weighted graph*).

Pada penelitian ini, lokasi-lokasi penting di lingkungan Universitas Bengkulu direpresentasikan sebagai node, sedangkan jalur penghubung antar lokasi direpresentasikan sebagai edge dengan bobot berupa jarak dalam satuan

meter. Representasi graf ini digunakan sebagai dasar dalam penerapan algoritma Dijkstra untuk menentukan jalur terpendek.

2.3 Algoritma Dijkstra

Algoritma Dijkstra merupakan algoritma yang digunakan untuk mencari jalur terpendek (*shortest path*) dari satu titik awal ke titik tujuan pada graf berbobot yang memiliki bobot non-negatif. Algoritma ini diperkenalkan oleh Edsger W. Dijkstra pada tahun 1959 dan hingga saat ini masih banyak digunakan dalam sistem navigasi, jaringan komputer, serta berbagai aplikasi pencarian rute (Dijkstra, 1959).

Prinsip kerja algoritma Dijkstra adalah memilih node dengan jarak terkecil dari titik awal, kemudian memperbarui jarak ke node-node tetangganya hingga seluruh node telah diperiksa atau tujuan telah ditemukan. Hasil akhir dari algoritma ini berupa jalur dengan total bobot terkecil yang dapat ditempuh dari titik awal menuju titik tujuan.

Pada penelitian ini, algoritma Dijkstra digunakan untuk menentukan jalur terpendek antar lokasi di lingkungan Universitas Bengkulu berdasarkan graf berbobot yang telah dibentuk dari data node dan edge. Bobot yang digunakan berupa jarak antar lokasi dalam satuan meter sehingga jalur yang dihasilkan merupakan rute dengan total jarak paling pendek.

2.4 OpenStreetMap (OSM)

OpenStreetMap (OSM) merupakan proyek pemetaan digital berbasis kolaborasi yang menyediakan data geografis secara bebas dan terbuka. Data pada OpenStreetMap dibuat dan diperbarui oleh komunitas pengguna di seluruh dunia sehingga dapat digunakan untuk berbagai kebutuhan, seperti navigasi, analisis spasial, dan pengembangan aplikasi berbasis lokasi (Haklay & Weber, 2008).

Salah satu keunggulan OpenStreetMap adalah kemampuannya menyediakan data peta yang dapat diakses secara gratis tanpa biaya lisensi. Selain itu, data yang

tersedia dapat diperbarui secara berkala oleh pengguna sehingga informasi yang ditampilkan tetap relevan dan akurat.

Pada penelitian ini, OpenStreetMap digunakan sebagai sumber data peta dasar yang menampilkan lokasi dan lingkungan Universitas Bengkulu. Data peta tersebut kemudian diintegrasikan dengan Leaflet untuk menampilkan visualisasi lokasi serta mendukung proses navigasi dan pencarian jalur terpendek pada sistem yang dikembangkan.

2.5 Leaflet.js

Leaflet.js merupakan library JavaScript open-source yang digunakan untuk membangun aplikasi peta interaktif berbasis web. Library ini dirancang agar ringan, mudah digunakan, serta mendukung berbagai sumber data peta seperti OpenStreetMap. Leaflet menyediakan berbagai fitur, seperti marker, popup, layer, dan visualisasi jalur yang memudahkan pengembang dalam membangun sistem informasi geografis berbasis web (Edler, 2019).

Leaflet banyak digunakan karena memiliki performa yang baik dan kompatibel dengan berbagai perangkat, baik komputer maupun perangkat mobile. Selain itu, Leaflet juga mendukung integrasi dengan berbagai layanan pemetaan dan routing sehingga cocok digunakan dalam pengembangan aplikasi navigasi.

Pada penelitian ini, Leaflet digunakan untuk menampilkan peta kampus Universitas Bengkulu, menampilkan marker lokasi, serta memvisualisasikan jalur terpendek yang dihasilkan oleh algoritma Dijkstra. Dengan menggunakan Leaflet, pengguna dapat berinteraksi langsung dengan peta untuk memilih lokasi asal dan tujuan perjalanan.

2.6 Open Source Routing Machine (OSRM)

Open Source Routing Machine (OSRM) merupakan layanan routing open-source yang digunakan untuk menghitung dan menghasilkan rute perjalanan berdasarkan data jaringan jalan dari OpenStreetMap. OSRM dirancang untuk

memberikan perhitungan rute yang cepat dan efisien sehingga banyak digunakan pada aplikasi navigasi dan sistem pencarian jalur (Luxen & Vetter, 2011).

OSRM mampu menghasilkan informasi rute berupa jarak tempuh, estimasi waktu perjalanan, serta koordinat jalur yang dapat ditampilkan pada peta digital. Dengan memanfaatkan data jalan yang tersedia pada OpenStreetMap, OSRM dapat menghasilkan visualisasi rute yang mengikuti kondisi jalan sebenarnya.

Pada penelitian ini, OSRM digunakan untuk memvisualisasikan jalur perjalanan yang telah ditentukan oleh algoritma Dijkstra. Hasil perhitungan jalur terpendek digunakan sebagai dasar untuk menentukan titik awal dan tujuan, kemudian OSRM menghasilkan koordinat rute yang ditampilkan pada peta menggunakan Leaflet sehingga pengguna dapat melihat jalur perjalanan secara lebih realistis.

BAB III

METODE PENELITIAN

3.1 Analisis Kebutuhan Sistem

Pada tahap ini dilakukan identifikasi kebutuhan sistem yang akan dibangun. Sistem navigasi kampus Universitas Bengkulu dirancang untuk membantu pengguna menemukan jalur terpendek menuju lokasi tujuan di lingkungan kampus. Sistem harus mampu menampilkan peta kampus, menerima input lokasi asal dan tujuan, menghitung jalur terpendek menggunakan algoritma Dijkstra, serta menampilkan hasil rute pada peta digital.

3.2 Pengumpulan Data

Data yang digunakan dalam penelitian ini berupa data lokasi gedung dan fasilitas yang terdapat di lingkungan Universitas Bengkulu. Data diperoleh melalui observasi langsung terhadap lingkungan kampus serta pemanfaatan data peta dari OpenStreetMap. Setiap lokasi dicatat koordinat geografisnya untuk dijadikan node dalam graf.

3.3 Perancangan Graf

Setelah data lokasi diperoleh, dilakukan pembentukan graf yang terdiri dari node dan edge. Node merepresentasikan lokasi atau gedung di lingkungan kampus, sedangkan edge merepresentasikan jalur yang menghubungkan dua lokasi. Setiap edge diberikan bobot berupa jarak antar lokasi dalam satuan meter.

3.4 Perancangan Sistem

Sistem dirancang menggunakan teknologi berbasis web dengan memanfaatkan HTML, CSS, dan JavaScript. Leaflet digunakan sebagai library pemetaan, OpenStreetMap sebagai penyedia data peta, serta Open Source Routing Machine (OSRM) sebagai layanan visualisasi rute. Algoritma Dijkstra digunakan untuk menentukan jalur terpendek berdasarkan graf yang telah dibentuk.

3.5 Implementasi Algoritma Dijkstra

Algoritma Dijkstra diterapkan pada graf yang telah dibangun untuk menghitung jalur terpendek dari titik asal menuju titik tujuan. Algoritma bekerja dengan mencari total bobot terkecil dari seluruh kemungkinan jalur yang tersedia hingga ditemukan rute terbaik.

3.6 Pengujian Sistem

Pengujian dilakukan dengan mencoba beberapa kombinasi lokasi asal dan tujuan pada lingkungan Universitas Bengkulu. Hasil pengujian kemudian diamati untuk memastikan bahwa sistem mampu menampilkan jalur terpendek, jarak tempuh, serta visualisasi rute sesuai dengan data graf yang telah dibentuk.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Pembangunan Sistem Navigasi Berbasis Web

Sistem navigasi kampus Universitas Bengkulu dikembangkan dalam bentuk aplikasi berbasis web yang dapat diakses melalui browser tanpa memerlukan instalasi perangkat lunak tambahan. Sistem ini dirancang untuk membantu pengguna dalam menentukan rute terpendek menuju lokasi yang diinginkan di lingkungan Universitas Bengkulu. Pengembangan sistem memanfaatkan teknologi web modern yang terdiri dari HTML sebagai struktur halaman, CSS sebagai pengatur tampilan antarmuka, serta JavaScript sebagai bahasa pemrograman utama untuk mengelola logika sistem dan interaksi pengguna.

Proses pembangunan sistem diawali dengan membuat elemen peta pada halaman web menggunakan HTML. Elemen tersebut berfungsi sebagai wadah utama untuk menampilkan peta digital. Setelah itu, JavaScript digunakan untuk menginisialisasi objek peta dan menentukan titik pusat (center) peta pada area Universitas Bengkulu. Selanjutnya, layer OpenStreetMap ditambahkan sehingga pengguna dapat melihat tampilan peta secara lengkap.

Berikut merupakan potongan kode yang digunakan untuk menginisialisasi peta dan menghubungkannya dengan OpenStreetMap.

```
const map = L.map('map', { center: [-3.7595, 102.2726], zoom: 16 });
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
  attribution: '© OpenStreetMap contributors',
  maxZoom: 21,
  maxNativeZoom: 19,
  detectRetina: true
}).addTo(map);
```

Gambar 4.1 Source code file app.js menghubungkan OpenStreetMap.

Pada kode tersebut, fungsi `L.map()` digunakan untuk membuat objek peta dengan titik pusat berada di lingkungan Universitas Bengkulu. Parameter `zoom` menentukan tingkat pembesaran awal peta saat pertama kali ditampilkan. Selanjutnya, fungsi `L.tileLayer()` digunakan untuk mengambil data peta dari

OpenStreetMap dan menampilkannya pada halaman web. Dengan demikian, pengguna dapat berinteraksi langsung dengan peta sebagai dasar proses pencarian jalur terpendek yang akan dibahas pada subbab berikutnya.

Arsitektur sistem secara umum terdiri dari pengguna sebagai pemberi input, aplikasi web sebagai media interaksi, data lokasi kampus yang direpresentasikan dalam bentuk graf, algoritma Dijkstra sebagai mesin pencari jalur terpendek, serta Open Source Routing Machine (OSRM) yang digunakan untuk menampilkan visualisasi rute pada peta digital. Hasil akhir dari proses tersebut berupa informasi jalur terpendek, jarak tempuh, estimasi waktu perjalanan, dan lokasi-lokasi yang dilewati selama perjalanan.

4.2 Representasi Lokasi Kampus dalam Bentuk Graf

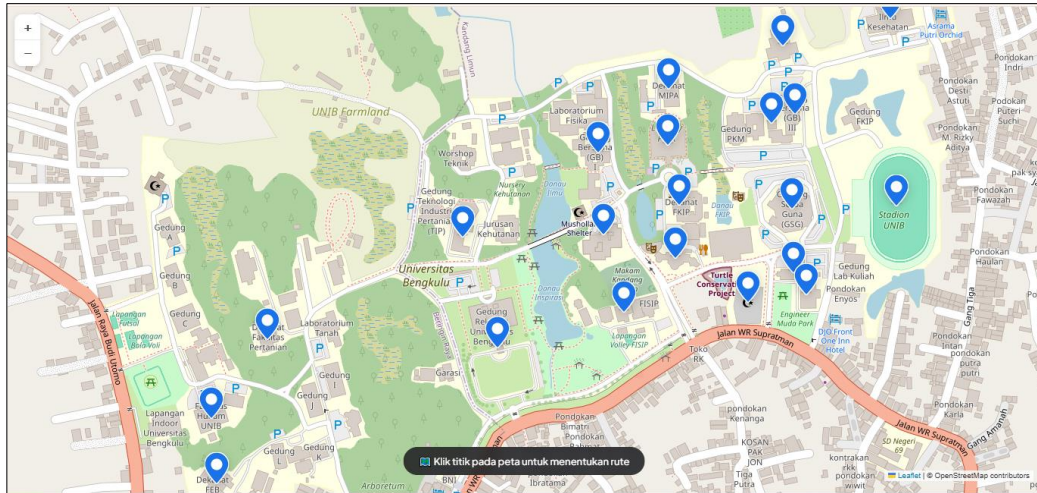
Untuk dapat menerapkan algoritma Dijkstra, lokasi-lokasi yang terdapat di lingkungan Universitas Bengkulu harus direpresentasikan terlebih dahulu ke dalam bentuk graf berbobot. Graf merupakan struktur data yang terdiri dari simpul (node) dan sisi (edge) yang saling terhubung. Pada penelitian ini, setiap lokasi penting di lingkungan kampus direpresentasikan sebagai node, sedangkan jalur yang menghubungkan dua lokasi direpresentasikan sebagai edge yang memiliki bobot berupa jarak tempuh dalam satuan meter.

Pada aplikasi yang dikembangkan, data lokasi kampus disimpan dalam variabel NODES. Setiap node memiliki identitas unik, nama lokasi, koordinat lintang (latitude), koordinat bujur (longitude), serta jenis ikon yang digunakan untuk ditampilkan pada peta. Beberapa lokasi yang direpresentasikan sebagai node antara lain Gedung Rektorat, LPTIK, Fakultas Teknik, Fakultas Pertanian, Perpustakaan, Masjid Baitul Hikmah, dan berbagai gedung lainnya yang berada di lingkungan Universitas Bengkulu.

Berikut merupakan sebagian potongan kode yang digunakan untuk mendefinisikan node pada sistem.

```
const NODES = {
  rektorat : { name: "Gedung Rektorat", lat: -3.7597, lng: 102.2724, iconType: "admin" },
  glt : { name: "Gedung Layanan Terpadu (GLT)", lat: -3.7581402, lng: 102.2719062, iconType: "office" },
  lptik : { name: "LPTIK UNIB", lat: -3.75843, lng: 102.27493, iconType: "tech" },
  kedokteran : { name: "Fakultas Kedokteran", lat: -3.75505, lng: 102.27797, routingLat: -3.75505, routingLng: 102.27797, iconType: "hospital" },
  mipa : { name: "Dekanat MIPA", lat: -3.7560482, lng: 102.2748329, iconType: "science" },
  teknik : { name: "Dekanat Teknik", lat: -3.75864, lng: 102.27660, routingLat: -3.75864, routingLng: 102.27660, iconType: "factory" },
  lab_terpadu : { name: "Laboratorium Teknik", lat: -3.75895, lng: 102.27678, routingLat: -3.75895, routingLng: 102.27678, iconType: "science" },
  stadion : { name: "Stadion UNIB", lat: -3.75770, lng: 102.27806, iconType: "stadium" },
  fpertanian : { name: "Dekanat Fakultas Pertanian", lat: -3.7595750, lng: 102.2691479, iconType: "agriculture" },
  fkip : { name: "Dekanat FKIP", lat: -3.75768, lng: 102.27498, iconType: "education" },
  fekon : { name: "Dekanat FEB", lat: -3.7616179, lng: 102.2684234, iconType: "economics" },
  perpustakaan : { name: "UPT Perpustakaan", lat: -3.7568392, lng: 102.2748198, iconType: "library" },
  gedung_bersama : { name: "Gedung Bersama (GB) II", lat: -3.75810, lng: 102.27391, iconType: "office" },
  masjid : { name: "Masjid Baitul Hikmah UNIB", lat: -3.7590596, lng: 102.2759543, iconType: "mosque" },
  fhukum : { name: "Fakultas Hukum UNIB", lat: -3.7606953, lng: 102.2683459, iconType: "law" },
  fisipol : { name: "Dekanat FISIP", lat: -3.7591964, lng: 102.2741934, iconType: "admin" },
  gb_v : { name: "Gedung Bersama (GB) V", lat: -3.75544, lng: 102.27645, iconType: "office" },
  gb_i : { name: "Gedung Bersama (GB) I", lat: -3.75695, lng: 102.27383, iconType: "office" },
  gb_iii : { name: "Gedung Bersama (GB) III", lat: -3.75640, lng: 102.27662, iconType: "office" },
  gb_iv : { name: "Gedung Bersama (GB) IV", lat: -3.75653, lng: 102.27629, iconType: "office" },
  gsg : { name: "Gedung Serba Guna (GSG)", lat: -3.75774, lng: 102.27658, iconType: "stadium" },
};
```

Gambar 4.2.1 Source code app.js bagian notes.



Gambar 4.2.2 Representasi nodes pada web.

Berdasarkan implementasi tersebut, setiap node menyimpan informasi lokasi yang nantinya digunakan untuk menampilkan marker pada peta serta menjadi titik yang dapat diproses oleh algoritma Dijkstra.

Selain node, sistem juga mendefinisikan hubungan antar lokasi dalam bentuk edge yang disimpan pada variabel EDGES. Setiap edge berisi informasi lokasi asal, lokasi tujuan, bobot jarak, serta jenis jalur yang dapat dilalui oleh pengguna. Bobot yang digunakan berupa jarak antar lokasi dalam satuan meter.

Berikut merupakan sebagian potongan kode yang digunakan untuk mendefinisikan edge.

```

39 const EDGES = [
40   // --- JALAN RAYA -- semua kendaraan ---
41   ["fekon", "fhukum", 100, "all"],
42   ["fhukum", "rektorat", 300, "all"],
43   ["rektorat", "glt", 180, "all"],
44   ["glt", "fpertanian", 340, "all"],
45   ["fpertanian", "rektorat", 260, "all"],
46   ["mipa", "kedokteran", 360, "all"],
47   ["fteknik", "kedokteran", 420, "all"],
48
49   // Rektorat & Gedung Bersama (GB)
50   ["rektorat", "gedung_bersama", 230, "all"],
51   ["gedung_bersama", "lptik", 120, "all"],
52   ["gedung_bersama", "fisipol", 130, "all"],
53   ["gb_i", "gedung_bersama", 130, "all"],
54   ["perpus", "gb_i", 110, "all"],
55
56   // Kampus Utara & Tengah
57   ["perpus", "mipa", 100, "all"],
58   ["perpus", "fkip", 100, "all"],
59   ["lptik", "fkip", 80, "all"],
60   ["lptik", "masjid", 130, "all"],
61   ["fkip", "masjid", 190, "all"],
62   ["fisipol", "masjid", 200, "all"],
63   ["fisipol", "stadion", 380, "all"],
64
65   // Kampus Timur (Fakultas Teknik & GB V, III, IV, GSG, Stadion)
66   ["gb_v", "gb_iii", 110, "all"],
67   ["gb_iii", "gb_iv", 40, "all"],
68   ["gb_iv", "gsg", 140, "all"],
69   ["gsg", "fteknik", 100, "all"],
70   ["fteknik", "lab_terpadu", 50, "all"],
71   ["mipa", "gb_iii", 200, "all"],
72   ["lptik", "fteknik", 180, "all"],
73   ["gsg", "stadion", 160, "all"],
74   ["kedokteran", "stadion", 290, "all"],
75
76   // --- GANG SEMPIT -- motor + jalan kaki ---
77   ["glt", "lptik", 220, "wm"],
78   ["gedung_bersama", "masjid", 240, "wm"],
79
80   // --- JALAN SETAPAK -- jalan kaki saja ---
81   ["glt", "lptik", 200, "w"],
82   ["gedung_bersama", "masjid", 220, "w"],
83   ["rektorat", "perpus", 300, "w"],
84   ["mipa", "gedung_bersama", 250, "w"],
85 ];
86

```

Gambar 4.2.3 Source code app.js bagian edges.

Pada kode tersebut, angka yang berada pada posisi ketiga menunjukkan bobot atau jarak antar node. Sebagai contoh, jalur antara Gedung Rektorat dan Gedung Layanan Terpadu (GLT) memiliki jarak 180 meter. Informasi ini digunakan oleh algoritma Dijkstra untuk menghitung total jarak yang harus ditempuh dari titik awal menuju titik tujuan.

Selain bobot jarak, sistem juga menerapkan klasifikasi jenis jalur berdasarkan moda transportasi. Terdapat tiga kategori jalur, yaitu all untuk semua kendaraan, wm untuk sepeda motor dan pejalan kaki, serta w untuk pejalan kaki saja. Dengan adanya klasifikasi ini, sistem dapat menghasilkan jalur yang berbeda sesuai dengan moda transportasi yang dipilih oleh pengguna.

4.3 Pembentukan Struktur Graph

Setelah seluruh lokasi kampus direpresentasikan ke dalam bentuk node dan edge, langkah berikutnya adalah membangun struktur graph yang dapat diproses oleh algoritma Dijkstra. Pada tahap ini, data node dan edge yang sebelumnya masih berupa daftar lokasi dan jalur diubah menjadi struktur adjacency list.

Struktur ini dipilih karena lebih efisien dalam menyimpan hubungan antar node dan memudahkan proses penelusuran jalur oleh algoritma.

Pada sistem yang dikembangkan, graph dibangun secara otomatis menggunakan data yang terdapat pada variabel NODES dan EDGES. Mula-mula sistem membuat objek graph kosong, kemudian setiap node diberikan daftar tetangga kosong. Setelah itu, seluruh data edge dibaca satu per satu dan dimasukkan ke dalam graph sebagai hubungan antar node.

Berikut merupakan potongan kode yang digunakan untuk membangun struktur graph.

```
const graph = {};  
for (const id in NODES) graph[id] = [];  
for (const [a, b, w, mode] of EDGES) {  
  const modes = parseModes(mode);  
  graph[a].push({ to: b, w, modes });  
  graph[b].push({ to: a, w, modes });  
}
```

Gambar 4.3 Source code app.js bagian struktur graph.

Melalui fungsi tersebut, sistem dapat membedakan jalur yang dapat dilalui oleh semua kendaraan, jalur yang hanya dapat dilalui oleh sepeda motor dan pejalan kaki, serta jalur yang khusus diperuntukkan bagi pejalan kaki. Dengan demikian, graph yang terbentuk tidak hanya menyimpan informasi jarak, tetapi juga menyimpan aturan penggunaan jalur berdasarkan moda transportasi. Hasil dari tahap ini adalah sebuah struktur graph berbobot yang berisi seluruh lokasi dan hubungan antar lokasi di lingkungan Universitas Bengkulu.

4.4 Implementasi Algoritma Dijkstra

Setelah struktur graph berhasil dibangun, tahap berikutnya adalah menerapkan algoritma Dijkstra untuk menentukan jalur terpendek antara titik asal dan titik tujuan. Algoritma Dijkstra merupakan salah satu algoritma shortest path yang digunakan untuk mencari lintasan dengan total bobot minimum pada graf berbobot non-negatif. Implementasi algoritma diawali dengan inisialisasi nilai jarak seluruh node menjadi tak hingga (Infinity). Kemudian jarak node awal diatur

menjadi nol karena node tersebut merupakan titik keberangkatan pengguna. Selain itu, sistem juga menyiapkan struktur data untuk menyimpan node sebelumnya (previous node) yang nantinya digunakan untuk membangun kembali jalur hasil pencarian.

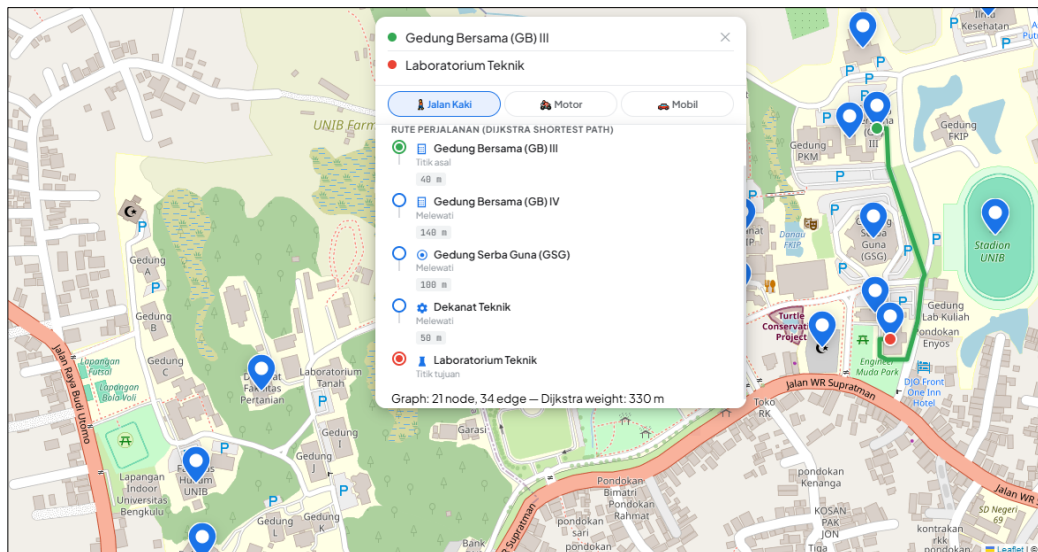
Berikut merupakan potongan kode inisialisasi algoritma Dijkstra.

```
function dijkstra(start, end, mode) {
  const dist = {}, prev = {}, visited = new Set();
  for (const id in NODES) { dist[id] = Infinity; prev[id] = null; }
  dist[start] = 0;
  const pq = [{ id: start, cost: 0 }];

  while (pq.length) {
    pq.sort((a, b) => a.cost - b.cost);
    const { id: u } = pq.shift();
    if (visited.has(u)) continue;
    visited.add(u);
    if (u === end) break;

    for (const { to: v, w, modes } of graph[u]) {
      if (!modes.includes(mode)) continue;
      const alt = dist[u] + w;
      if (alt < dist[v]) {
        dist[v] = alt;
        prev[v] = u;
        pq.push({ id: v, cost: alt });
      }
    }
  }
}
```

Gambar 4.4.1 Source code app.js bagian algoritma Dijkstra.



Gambar 4.4.2 Implementasi algoritma Dijkstra pada web.

Pada implementasi tersebut, sistem juga mempertimbangkan moda transportasi yang dipilih oleh pengguna. Jalur yang tidak sesuai dengan moda transportasi akan diabaikan sehingga hasil pencarian tetap sesuai dengan kondisi sebenarnya. Sebagai contoh, jalur yang hanya diperuntukkan bagi pejalan kaki tidak akan diproses ketika pengguna memilih moda transportasi mobil.

Setelah seluruh proses pencarian selesai, algoritma melakukan rekonstruksi jalur dengan menelusuri kembali node-node yang tersimpan pada variabel `prev`. Jalur tersebut kemudian disusun dari titik awal hingga titik tujuan dan dikembalikan sebagai hasil pencarian.

Berikut merupakan potongan kode yang digunakan untuk membentuk kembali jalur hasil pencarian.

```
if (dist[end] === Infinity) return null;
const path = [];
let cur = end;
while (cur) { path.unshift(cur); cur = prev[cur]; }
return { distance: dist[end], path };
}
```

Gambar 4.4.3 Source code `app.js` membentuk jalur hasil pencarian kembali.

Hasil dari proses ini berupa total jarak terpendek serta urutan lokasi yang harus dilalui oleh pengguna untuk mencapai tujuan. Informasi tersebut kemudian digunakan pada tahap berikutnya untuk menampilkan visualisasi rute pada peta digital. Dengan demikian, algoritma Dijkstra berperan sebagai komponen utama dalam menentukan rute terbaik pada sistem navigasi kampus Universitas Bengkulu.

4.5 Visualisasi Jalur Terpendek Pada Peta Digital

Setelah algoritma Dijkstra berhasil menentukan jalur terpendek antara titik asal dan titik tujuan, langkah berikutnya adalah menampilkan hasil perhitungan tersebut dalam bentuk visual yang mudah dipahami oleh pengguna. Pada sistem UNIB Maps, visualisasi rute dilakukan menggunakan library Leaflet yang terintegrasi dengan layanan Open Source Routing Machine (OSRM). Integrasi ini

memungkinkan sistem menampilkan jalur perjalanan secara lebih realistis mengikuti bentuk jalan yang sebenarnya di lingkungan kampus.

Proses visualisasi dimulai setelah pengguna memilih titik asal dan titik tujuan. Sistem kemudian menjalankan algoritma Dijkstra untuk memperoleh urutan node yang membentuk jalur terpendek. Hasil tersebut selanjutnya digunakan sebagai dasar untuk melakukan permintaan data rute kepada layanan OSRM. Layanan ini akan menghasilkan koordinat jalur yang mengikuti kondisi jalan actual sehingga tampilan rute menjadi lebih akurat dibandingkan sekadar menghubungkan titik-titik lokasi menggunakan garis lurus.

Berikut merupakan potongan kode yang digunakan untuk mengambil data rute dari layanan OSRM.

```
const coords = `${startLng},${startLat};${endLng},${endLat}`;  
const url = `https://router.project-osrm.org/route/v1/${cfg.profile}/${coords}?overview=f  
  
let latlngs, osrmDist, osrmDuration;  
const speeds = { walk: 1.3, motor: 8.3, car: 5.6 }; // Kecepatan realistis m/detik  
  
try {  
  const resp = await fetch(url);  
  const data = await resp.json();
```

Gambar 4.5.1 Source code app.js mengambil data rute layanan OSRM.

Pada kode tersebut, sistem mengirimkan koordinat titik awal dan titik tujuan ke layanan OSRM. Selanjutnya OSRM mengembalikan data rute dalam format GeoJSON yang berisi kumpulan koordinat jalur yang akan digambar pada peta. Setelah koordinat rute diperoleh, sistem melakukan proses visualisasi menggunakan objek polyline yang disediakan oleh Leaflet. Polyline digunakan untuk menggambar garis yang menghubungkan seluruh titik koordinat hasil perhitungan sehingga membentuk jalur perjalanan yang dapat dilihat oleh pengguna. Berikut merupakan potongan kode yang digunakan untuk menggambar jalur pada peta.

```
const shadow = L.polyline(visuallatlngs, { color: cfg.color, weight: 9, opacity: .15 }).addTo(map);  
const line = L.polyline(visuallatlngs, { color: cfg.color, weight: 5, opacity: 1,  
  lineJoin: 'round', lineCap: 'round' }).addTo(map);  
pathLayers = [shadow, line];
```

Gambar 4.5.2 Source code app.js menggambar jalur peta.

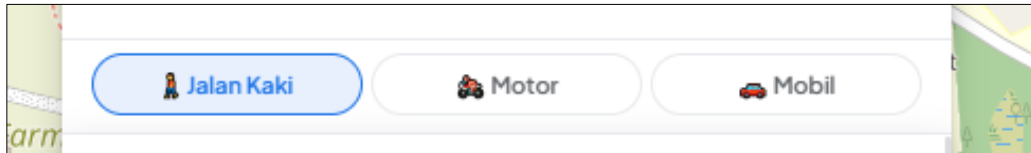
4.6 Implementasi Mode Transportasi

Sistem navigasi yang dikembangkan tidak hanya mampu menentukan jalur terpendek berdasarkan jarak, tetapi juga mempertimbangkan jenis moda transportasi yang digunakan oleh pengguna. Fitur ini diterapkan untuk menghasilkan rute yang lebih sesuai dengan kondisi jalur yang tersedia di lingkungan Universitas Bengkulu. Implementasi mode transportasi dilakukan melalui konfigurasi yang menentukan profil perjalanan untuk setiap jenis kendaraan. Konfigurasi tersebut digunakan untuk mengatur proses pencarian rute dan visualisasi jalur pada peta.

Berikut merupakan potongan kode yang digunakan untuk mendefinisikan mode transportasi pada sistem.

```
const MODE_CONFIG = {  
  walk: { profile: 'foot', label: 'Jalan Kaki', color: '#34a853' },  
  motor: { profile: 'bicycle', label: 'Motor', color: '#1a73e8' },  
  car: { profile: 'driving', label: 'Mobil', color: '#ea4335' },  
};  
let currentMode = 'walk';
```

Gambar 4.6.1 Source code app.js bagian mode transportasi.



Gambar 4.6.2 Jenis transportasi pada web.

Selain konfigurasi mode, sistem juga menerapkan klasifikasi jalur berdasarkan jenis kendaraan yang diperbolehkan untuk melintasinya. Klasifikasi tersebut terdiri dari tiga kategori, yaitu all, wm, dan w.

Berikut merupakan potongan kode yang digunakan untuk menentukan akses moda transportasi pada setiap jalur.

```
function parseModes(m) {  
  if (m === 'all') return ['walk', 'motor', 'car'];  
  if (m === 'wm') return ['walk', 'motor'];  
  return ['walk'];  
}
```

Gambar 4.6.2 Source code app.js menentukan jalur transportasi

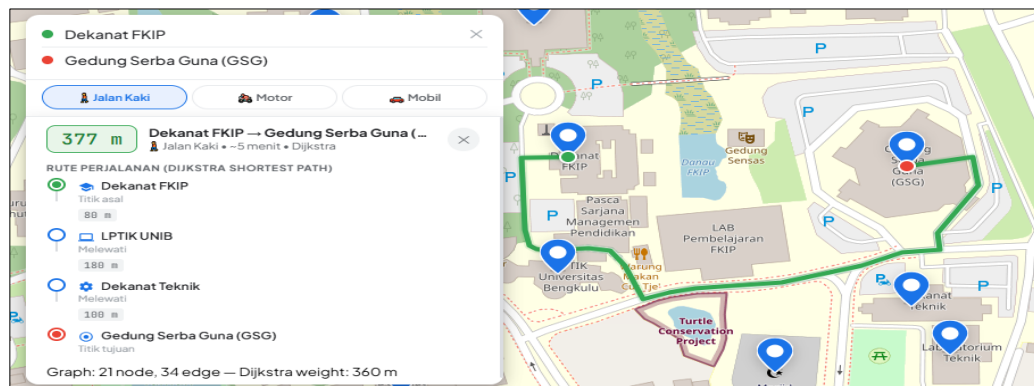
Pada implementasi tersebut, kategori all menunjukkan jalur yang dapat digunakan oleh semua moda transportasi. Kategori wm menunjukkan jalur yang hanya dapat digunakan oleh pejalan kaki dan sepeda motor, sedangkan kategori w menunjukkan jalur yang hanya dapat digunakan oleh pejalan kaki.

Ketika algoritma Dijkstra melakukan proses pencarian jalur, sistem akan memeriksa terlebih dahulu apakah suatu edge dapat dilalui oleh moda transportasi yang dipilih pengguna. Jika tidak sesuai, edge tersebut akan diabaikan sehingga tidak ikut dipertimbangkan dalam proses pencarian rute.

Dengan mekanisme tersebut, sistem mampu menghasilkan jalur yang berbeda sesuai dengan moda transportasi yang digunakan. Hal ini membuat hasil navigasi menjadi lebih realistis dan sesuai dengan kondisi akses jalan yang terdapat di lingkungan Universitas Bengkulu.

4.7 Hasil Pengujian Sistem

Pengujian sistem dilakukan untuk mengetahui apakah aplikasi navigasi kampus Universitas Bengkulu yang dikembangkan dapat berfungsi sesuai dengan tujuan yang telah ditetapkan. Pengujian dilakukan dengan mencoba beberapa kombinasi titik asal dan titik tujuan pada lokasi yang terdapat di lingkungan kampus. Setiap pengujian bertujuan untuk memastikan bahwa algoritma Dijkstra mampu menentukan jalur terpendek dan sistem dapat menampilkan hasil rute secara benar pada peta digital.



Gambar 4.7.1 Pengujian rute.

Hasil pengujian menunjukkan bahwa sistem berhasil menentukan jalur terpendek berdasarkan graf yang telah dibentuk. Selain itu, sistem juga mampu menampilkan informasi jarak tempuh, urutan lokasi yang dilewati, serta visualisasi rute pada peta secara interaktif.

No	Titik asal	Titik Tujuan	Hasil
1	Dekanat FKIP	Gedung Serba Guna (GSG)	Berhasil
2	Gedung Rektorat	Dekanat Teknik	Berhasil
3	Fakultas Hukum	UPT Perpustakaan	Berhasil
4	Dekanat FEB	Fakultas Kedokteran	Berhasil
5	Masjid Baitul Hikmah	Stadion UNIB	Berhasil

Tabel 4.7 Pengujian yang dilakukan pada sistem.

Pada setiap pengujian, sistem mampu menghasilkan rute yang sesuai dengan jalur yang tersedia pada graf. Hasil pengujian juga menunjukkan bahwa algoritma Dijkstra dapat menemukan jalur dengan total bobot terkecil berdasarkan data jarak yang telah ditentukan pada setiap edge. Sebagai contoh, pada pengujian rute dari Dekanat FKIP menuju Gedung Serba Guna (GSG), sistem menghasilkan jalur melalui LPTIK dan Dekanat Teknik dengan total bobot sebesar 360 meter. Jalur tersebut merupakan jalur dengan total jarak paling kecil dibandingkan jalur alternatif lainnya yang tersedia pada graf.

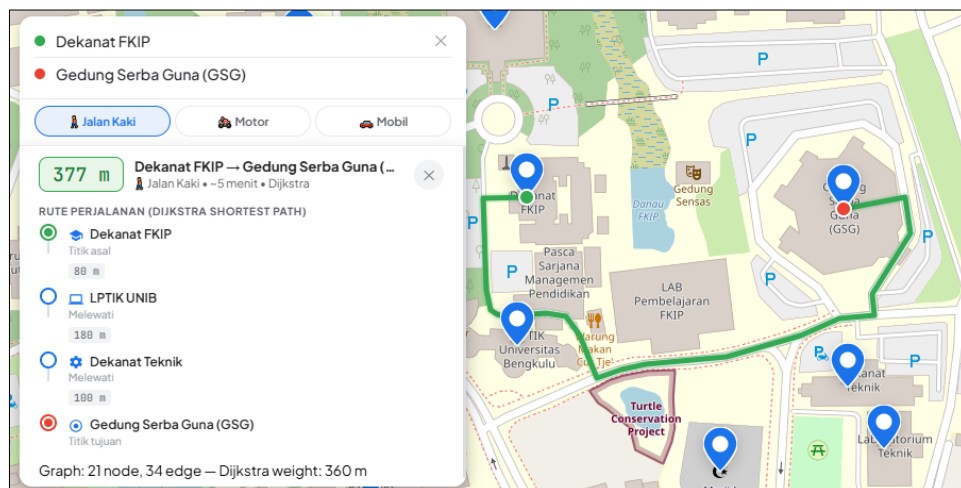
Selain pengujian jalur, dilakukan pula pengujian terhadap fitur pemilihan moda transportasi. Hasil pengujian menunjukkan bahwa sistem dapat menyesuaikan jalur yang direkomendasikan berdasarkan jenis kendaraan yang dipilih oleh pengguna. Jalur yang tidak sesuai dengan moda transportasi akan diabaikan selama proses pencarian rute sehingga hasil yang diperoleh lebih sesuai dengan kondisi akses jalan yang sebenarnya. Secara keseluruhan, hasil pengujian menunjukkan bahwa sistem navigasi kampus Universitas Bengkulu telah berjalan dengan baik. Seluruh fitur utama, mulai dari pemilihan lokasi, pencarian jalur terpendek menggunakan algoritma Dijkstra, visualisasi rute pada peta, hingga

pemilihan moda transportasi, dapat berfungsi sesuai dengan rancangan yang telah dibuat.

4.8 Analisis Hasil

Analisis hasil dilakukan untuk mengevaluasi kinerja sistem navigasi kampus Universitas Bengkulu yang telah dikembangkan. Analisis ini mencakup kemampuan algoritma Dijkstra dalam menentukan jalur terpendek, pengaruh mode transportasi terhadap hasil navigasi, serta kualitas visualisasi rute yang ditampilkan pada peta digital. Hasil analisis digunakan untuk mengetahui kelebihan dan keterbatasan sistem yang telah dibangun.

4.8.1 Analisis Kinerja Algoritma Dijkstra



Gambar 4.8.1 Pengujian rute Dekanat FKIP – GSG.

Berdasarkan hasil pengujian yang telah dilakukan, algoritma Dijkstra berhasil menentukan jalur terpendek antara titik asal dan titik tujuan pada lingkungan Universitas Bengkulu. Algoritma bekerja dengan memanfaatkan graf berbobot yang terdiri dari node sebagai lokasi dan edge sebagai jalur penghubung antar lokasi.

Hasil yang diperoleh menunjukkan bahwa sistem mampu memilih jalur dengan total bobot terkecil dibandingkan jalur alternatif yang tersedia. Sebagai contoh, pada rute dari Dekanat FKIP menuju Gedung Serba Guna

(GSG), sistem menghasilkan jalur melalui LPTIK dan Dekanat Teknik dengan total bobot sebesar 360 meter. Jalur tersebut dipilih karena memiliki jarak yang lebih pendek dibandingkan jalur lainnya.

Selain itu, algoritma juga mampu menyesuaikan proses pencarian berdasarkan moda transportasi yang dipilih pengguna dengan mengabaikan edge yang tidak dapat dilalui oleh kendaraan tertentu. Hal ini menunjukkan bahwa implementasi algoritma Dijkstra pada sistem telah berjalan sesuai dengan konsep shortest path dan mampu menghasilkan rute yang optimal berdasarkan data graf yang digunakan.

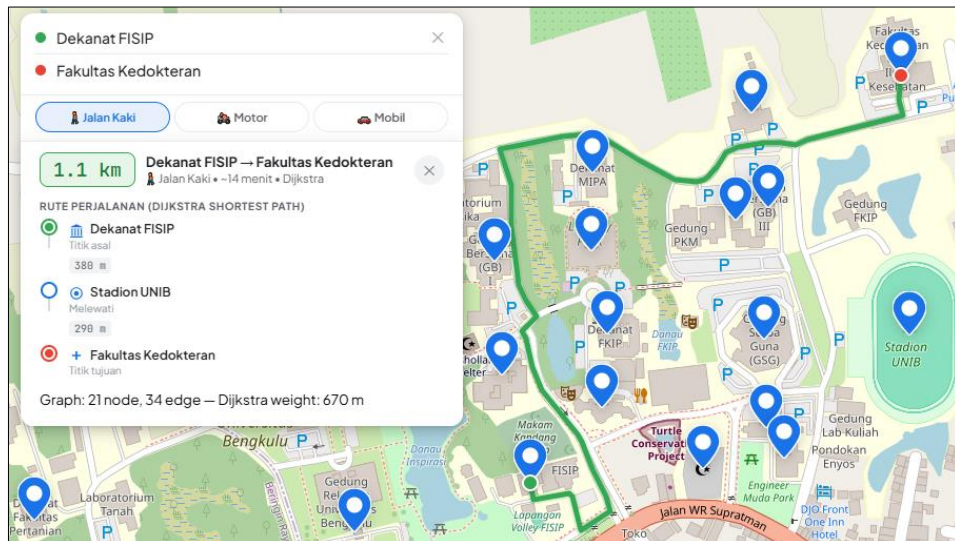
4.8.2 Analisis Pengaruh Mode Transportasi

Sistem navigasi yang dikembangkan menyediakan tiga pilihan mode transportasi, yaitu jalan kaki, sepeda motor, dan mobil. Setiap mode transportasi memiliki akses yang berbeda terhadap jalur yang tersedia pada graf. Perbedaan ini menyebabkan hasil rute yang diperoleh dapat berbeda meskipun titik asal dan tujuan yang dipilih sama.

Pada mode jalan kaki, sistem dapat menggunakan seluruh jenis jalur, termasuk jalan setapak dan gang sempit. Pada mode sepeda motor, sistem hanya menggunakan jalur yang dapat dilalui kendaraan roda dua, sedangkan pada mode mobil sistem hanya menggunakan jalur utama yang dapat dilalui kendaraan roda empat.

Berdasarkan hasil pengujian, fitur mode transportasi berhasil berfungsi sesuai dengan rancangan sistem. Algoritma Dijkstra hanya memproses edge yang sesuai dengan mode transportasi yang dipilih sehingga rute yang dihasilkan menjadi lebih realistis dan sesuai dengan kondisi akses jalan yang sebenarnya. Dengan demikian, pengguna dapat memperoleh rekomendasi jalur yang lebih akurat sesuai dengan kendaraan yang digunakan.

4.8.3 Analisis Visualisasi Rute



Gambar 4.8.3 Visualisasi Route.

Visualisasi rute pada sistem berhasil menampilkan hasil pencarian jalur terpendek dalam bentuk yang mudah dipahami oleh pengguna. Jalur yang diperoleh dari proses perhitungan kemudian ditampilkan pada peta digital menggunakan Leaflet dan layanan OSRM sehingga mengikuti bentuk jalan yang tersedia pada area kampus.

Selain menampilkan garis rute pada peta, sistem juga menampilkan informasi tambahan berupa jarak tempuh, estimasi waktu perjalanan, serta urutan lokasi yang dilalui. Informasi tersebut membantu pengguna dalam memahami rute yang direkomendasikan dan memudahkan proses navigasi menuju lokasi tujuan.

Berdasarkan hasil pengujian, visualisasi rute yang ditampilkan telah sesuai dengan hasil perhitungan algoritma Dijkstra dan mampu memberikan informasi perjalanan secara jelas kepada pengguna.

4.8.4 Kelebihan dan Keterbatasan Sistem

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, sistem memiliki beberapa kelebihan. Sistem dapat menentukan jalur terpendek menggunakan algoritma Dijkstra, menampilkan rute secara

interaktif pada peta digital, serta mendukung beberapa mode transportasi yang berbeda. Selain itu, aplikasi dapat diakses melalui web browser tanpa memerlukan proses instalasi tambahan.

Meskipun demikian, sistem masih memiliki beberapa keterbatasan. Data lokasi yang digunakan masih terbatas pada titik-titik yang telah dimasukkan ke dalam graf sehingga belum mencakup seluruh area kampus secara detail. Selain itu, bobot jarak pada beberapa edge masih ditentukan secara manual dan sistem belum mempertimbangkan faktor dinamis seperti kondisi lalu lintas, kepadatan kendaraan, atau perubahan akses jalan secara real-time.

Secara keseluruhan, sistem yang dikembangkan telah mampu memenuhi tujuan utama penelitian, yaitu menyediakan layanan navigasi kampus berbasis web yang dapat membantu pengguna menemukan jalur terpendek menuju lokasi yang diinginkan di lingkungan Universitas Bengkulu.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem navigasi kampus Universitas Bengkulu berbasis web berhasil dikembangkan dengan memanfaatkan algoritma Dijkstra untuk menentukan jalur terpendek antar lokasi di lingkungan kampus. Sistem mampu merepresentasikan lokasi kampus dalam bentuk graf yang terdiri dari node dan edge sehingga proses pencarian rute dapat dilakukan secara efektif.

Hasil pengujian menunjukkan bahwa algoritma Dijkstra mampu menghasilkan jalur dengan total bobot terkecil sesuai dengan data graf yang telah dibentuk. Selain itu, sistem juga berhasil menampilkan visualisasi rute pada peta digital menggunakan Leaflet, OpenStreetMap, dan OSRM sehingga pengguna dapat melihat jalur perjalanan secara lebih jelas dan interaktif.

Fitur mode transportasi yang terdiri dari jalan kaki, sepeda motor, dan mobil juga berhasil diterapkan sehingga rute yang dihasilkan dapat menyesuaikan dengan jenis kendaraan yang digunakan. Secara keseluruhan, sistem yang dikembangkan telah memenuhi tujuan penelitian, yaitu menyediakan layanan navigasi kampus yang mampu membantu pengguna menemukan jalur terpendek menuju lokasi yang diinginkan di lingkungan Universitas Bengkulu.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan sistem di masa mendatang, yaitu :

1. Menambahkan lebih banyak lokasi dan jalur pada graf agar cakupan area kampus menjadi lebih lengkap.
2. Menggunakan data jarak yang diperoleh secara otomatis dari peta digital sehingga bobot edge menjadi lebih akurat.

3. Menambahkan fitur penentuan lokasi pengguna secara otomatis menggunakan GPS.
4. Mengembangkan sistem agar dapat memberikan petunjuk navigasi secara real-time selama perjalanan.
5. Menambahkan informasi fasilitas pendukung seperti area parkir, kantin, ATM, dan fasilitas umum lainnya untuk meningkatkan manfaat aplikasi bagi pengguna.

Dengan pengembangan lebih lanjut, sistem navigasi kampus ini diharapkan dapat menjadi sarana informasi dan navigasi yang lebih lengkap serta bermanfaat bagi civitas akademika Universitas Bengkulu.

DAFTAR PUSTAKA

- Burrough, P. A., & McDonnell, R. A. (1998). Principles of Geographical Information Systems. Oxford University Press.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
- Edler, D. (2019). An Example with the Open-Source JavaScript Library Leaflet.js. *KN - Journal of Cartography and Geographic Information*.
- Gross, J. L., & Yellen, J. (2006). Graph Theory and Its Applications (2nd ed.). Chapman & Hall/CRC.
- Haklay, M., & Weber, P. (2008). OpenStreetMap: User-Generated Street Maps. *IEEE Pervasive Computing*, 7(4), 12–18.
- Ngugi, S. (2023). Leaflet in Practice: Create Webmaps Using the JavaScript Leaflet Library.
- Zhan, F. B., & Noon, C. E. (1998). Shortest Path Algorithms: An Evaluation Using Real Road Networks. *Transportation Science*, 32(1), 65–73.